

The Expression Evaluator

Over the course of the semester, we're going to write an interpreter for a non-standard program language. We need to cover some class material before we start parsing, so we're going to start with a simple (or maybe not-so-simple) expression evaluator.

You will implement an expression evaluator in Racket. The evaluator will consume Abstract Syntax Trees (ASTs) represented as S-expressions—the exact structural output your future parser will produce. The goal of this assignment is to understand the mechanics of tree-walking evaluation, environment manipulation (variable lookups), and runtime semantic enforcement. However, there is a fundamental twist: this language does not operate on binary logic.

Kleene's Three-Valued Logic (3VL)

To reflect the messy reality of runtime environments where data can be missing, corrupt, or uninitialized, our language introduces a third truth value. In addition to the standard boolean literals yes (True) and no (False), your evaluator must support a third primitive literal: maybe (representing an Unknown, Undefined, or Erroneous state).

Rather than crashing the interpreter when an error occurs, your evaluator must gracefully degrade the evaluation branch to 'maybe and propagate it through the execution tree using Kleene's Three-Valued Logic matrix.

The Logic Tables

The core intuition of 3VL is that a 'maybe value is simply a placeholder for a concrete value that is not yet known. Therefore, operations must be evaluated based on whether a definitive conclusion can be reached *despite* the ambiguity.

- Conjunction (and): False ('no) dominates. If one side of an and expression is 'no, the entire expression is 'no, even if the other side is completely unknown ('maybe). Conversely, if one side is 'yes and the other is 'maybe, the result remains 'maybe.

- Disjunction (or): True ('yes) dominates. If one side of an or expression is 'yes, the entire expression is 'yes, regardless of whether the other side is 'maybe.

Your evaluator must strictly adhere to the following truth tables for all logical operations:

Table 1: Truth values for 3-valued logic

Left operand	Operator	Right operand	Result
'yes	and	'maybe	'maybe
'no	and	'maybe	'no
'maybe	and	'maybe	'maybe
'yes	or	'maybe	'yes
'no	or	'maybe	'maybe
'maybe	or	'maybe	'maybe
N/A	not	'maybe	'maybe

Note that Racket's built-in 'and', 'or', and 'not' functions do not handle maybe; you will need to write your own in accordance with the above table.

Your evaluator must intercept potential runtime exceptions before they hit the underlying Racket architecture. If any of the following scenarios occur during execution, the offending sub-expression must immediately evaluate to 'maybe:

1. Division by Zero: Any numeric expression attempting to divide a number by 0 (e.g., 5 / 0).
2. Type Mismatch: Any operation supplied with incompatible types (e.g., adding an integer to a string, or performing a logical and on two real numbers).
3. Unbound Variable: Attempting to look up an identifier that does not exist within the current environment.

Once a sub-expression degrades to 'maybe, that value must ripple upward and interact mathematically or logically with the rest of the AST according to the 3VL matrices above.

Architectural Requirements

To receive full credit, your solution must be structured strictly around functional programming paradigms:

- No Global State: You may not use global variables, mutation (set!, hash-set!), or Racket's internal eval function.

- **Total Functions:** Your eval-expr function must be total; it should return a valid symbol ('yes', 'no', 'maybe'), string, or number for *all* syntactically valid AST inputs, never throwing an unhandled Racket runtime error.
- **Final form:** Since some expression evaluations might return an expression that itself needs evaluating, or contain sub-expressions that need evaluating, when your evaluation function returns something, if what it returns is different from what it was passed, you should call your evaluation function again on what was returned. It's not completely evaluated until what's returned is the same as what was passed to it.

Expression format

Your Racket evaluator will expect the expression to be passed in as a nested list structure (S-expression). This format cleanly mirrors the format of the parser you will write in program 2.

Here is the structural specification for each type of node your evaluator must handle.

1. Primitives and Literals
 - Literal values are wrapped in a tagged list denoting their category.
 - Integers & Reals: '(lit <number>) (e.g., '(lit 5) or '(lit 3.14))
 - Booleans: '(lit yes) or '(lit no) or '(lit maybe)
 - Strings: '(lit <string>) (e.g., '(lit "Hello"))
2. Identifiers (Variables)
 - Variable lookups are represented by the tag var followed by the string identifier.
 - a. Format: '(var <string>) (e.g., '(var "x"))
3. Unary Operations
 - The only unary operation in the grammar is logical negation.
 - a. Format: '(not <expression>)
 - b. Example (not yes): '(not (lit yes))
4. Binary Operations
 - All binary calculations, comparisons, and logical operations share a unified structural format: '(binary-op <operator-string> <left-expression> <right-expression>).

Operators Handled

Your evaluator must handle the following operator strings across their respective categories:

Table 2: Operators and Types

Category	Operator Strings	Expected Types
Arithmetic	"+" , "-" , "*" , "/"	Numbers (Integer or Real)
String Manipulation	"~" (<i>Concatenation</i>)	Strings
Relational	"==" , "/=" , "<" , "<=" , ">" , ">="	Numbers or Strings
Logical	"and" , "or"	3VL Symbols ('yes, 'no, 'maybe)

Examples

Example A: Division by Zero

Consider this expression containing a potential division-by-zero error:

```
(x + 10) / (y * 0)
```

The parser will output this deeply nested S-expression, which your evaluator must walk recursively:

```
'(binary-op "/"
  (binary-op "+" (var "x") (lit 10))
  (binary-op "*" (var "y") (lit 0)))
```

Example B: Mixed Logical and Relational Operations

Consider a conditional boundary check:

```
(score >= 75.0) and not flag
```

This will translate into:

```
'(binary-op "and"
  (binary-op ">=" (var "score") (lit 75.0))
  (not (var "flag")))
```

Example C: Combined Expressions

6A – 3A translates into

```
(binary-op "-"
  '(binary-op "*" (lit 6) (var "A"))
  '(binary-op "*" (lit 3) (var "A")))
```

The Evaluator Function Signature

Here is the basic outline of your evaluator. Your function will take an expression, and a Racket hashmap representing the environment. Under some conditions, it is possible that an expression will ‘evaluate’ as a simpler expression. In that case, the returned expression should be sent through the evaluator on its own. This should continue until either it is reduced to a literal value (which cannot be

simplified further) or the returned expression is identical to the expression passed in. For any expressions containing a variable, the environment hash should be checked for that value. If it has a value, replace the variable in the expression with the literal value, and re-evaluate. If the variable is not defined, or the types are not compatible, or involves division by 0, the value of the expression is `'(lit maybe)`.

Your evaluation function should match this signature, accepting the S-expression and an immutable Racket hash map acting as the environment:

```
(define (eval-expr expr env)
  (match ast
    [`(lit val)                ...]
    [`(var id)                 ...]
    [`(not expr)               ...]
    [`(binary-op op left right) ...]
    [else (error "Invalid AST node structure")]))
```

Test cases

You have a file attached to the assignment with several test cases to test the various identities and 3-value logic expressions.

Deliverables

You will submit the following:

- A Racket code (.rkt) file with your expression evaluator, along with whatever support/test code you need.
- Logs of your AI interactions. This can be saving the interactions to a document, or a link to the session on the internet.
- A 7-10 minute video recorded using Panopto, or using Zoom and uploading it.
 - It should include a view of your screen, and your face must be visible on the video at all times (the video, not just a pic of you). The narration must be in your voice, not text-to-speech. Your video should demonstrate that your code runs and produces correct output. Then discuss how you handled and tested the 3-valued logic. Then discuss any portions of the code that gave you trouble, that turned out to be easier than you expected, or that you're particularly proud of. That should take up about the first half of the video.

- The second half concerns your interaction with the AI itself and your development and testing process. Choose one or more of these questions, whichever you find most interesting or think you have the most to say:
 - Where was AI most helpful? Where was it least helpful?
 - Which parts of the assignment were most challenging for *you*? Most interesting (which isn't necessarily the same thing)?
 - How did you test your code and verify correctness? Give one or more examples of where the AI code didn't work and what *you* did to fix it.
 - Were there any parts of the assignment you didn't think you'd be able to do? How did you find a workaround or get them done?
 - Did the AI suggest Racket functions that don't actually exist? Did you have to reject a solution because it was horribly inefficient?
 - Did using AI help you understand the language or concepts like tail recursion better, or did it make it more confusing?
 - Any other questions about the assignment or process that you found interesting or think worth discussing.

Grading Rubric

Table 3: Grading Rubric

Content	Excellent	Acceptable	Unacceptable
Functionality (35%). <i>Does the program correctly transform the trees according to the stated rules?</i>	Successfully reaches a fixed point for all test cases. Correctly simplifies deeply nested identities and constant folds across branches.	Correctly handles Level 1 & 2. Fails on "Deep simplification" (fixed point) or specific nested identities like '(* 1 (+ x 0))	Fails on basic constant folding. Program crashes on nested structures or unrecognized symbols.
Video: Understanding & Logic (25%) <i>Can the student explain the mechanics of tree traversal?</i>	Articulates the post-order nature of the traversal. Clearly explains how the pattern matcher identifies specific tree	Traces a basic recursive call accurately. Understands that children must be simplified before the parent node can	Cannot explain how the code distinguishes between a list (expression) and a symbol (variable). Confused by the recursive return value. -

	'shapes' to apply rules.	be reduced, and can explain why this is so. Racket screen & student's face visible throughout the video. Narration is in student's voice.	10% to -15% if face/screen requirements are not met.
Video: Reflection & Analysis (15%) <i>Connection to programming language design concepts.</i>	Discusses the benefit of symbolic manipulation over string parsing. Compares Racket's match capabilities to switch or if/else logic. Analysis of AI interaction.	Identifies that the S-expression represents a tree. Explains why this is an "optimization" phase for a language. Some analysis or reflection on AI strengths & weaknesses.	Views the project as a math problem only. No mention of ASTs, IR (Intermediate Representation), or compiler phases. Reflection is vague, nonspecific, surface level only. (-10% for things like "the AI was helpful and I liked using it.")
Documentation: AI Logs (15%) <i>The 'paper trail' of how AI was used, the basis of the audit log.</i>	Logs show a "Co-pilot" relationship. Student caught AI errors in recursion or asked the AI to refactor imperative code into something more readable. Clear path from prompt to final modified code.	Logs show the student used AI to overcome Racket syntax hurdles (e.g., "How do I check if something is a symbol?"). Multiple prompts used but some appear to be missing. Student identifies a mistake made by AI but struggles to fix it without more AI help. Path from prompt to final code is unclear or missing.	Logs show "blind copying." No evidence of the student asking "Why does this work?" or "Explain this syntax." One-shot prompt (e.g. "write this program") No evidence of checking for errors or evidence of iterative process. If AI code is much different from final product, no discussion of the process or evidence of iterative process. -10% for 'sanitized' logs.
Code Quality & Style (10%) <i>Readability,</i>	Elegant use of pattern matching. Code reads like a	Uses match or cond clearly. Indentation follows	Code is a "wall of text." Variable names are non-descriptive. No

<i>naming conventions, & structural clarity.</i>	formal specification of the algebraic rules. Zero redundant logic; high "signal-to-noise" ratio. Appropriate use of let, let*, and helper functions. Clean indentation reflecting recursive structure. Clear naming & use of comments.	standard Racket conventions. Code is modular (e.g., separate functions for folding and reducing). Mostly clear naming conventions. Some redundant code but not much. Some signs of 'AI fingerprints' but not enough to cause problems with readability or intelligibility.	comments explaining the algebraic rules being applied. Messy indentation. Deeply nested if statements rather than cond or match.
--	---	---	---